AI Evaluation Lab

Project overview

Compare deterministic assistant candidates against a small regression dataset and inspect correctness, refusal behavior, and latency.

This guide explains the delivered implementation, how to run it, what each component does, the algorithms, and the limits of the demonstration.

Prepared for AI Product Reviewer / Beta-Testing internship interviews. Synthetic data. Local demo. AI-assisted starter; personalize and understand before presenting.

AI Evaluation Lab

Problem and demo

AI products can regress when prompts or models change.

This app makes a tiny, repeatable evaluation workflow visible.

Select baseline, run the suite, and inspect the failed secret-disclosure refusal case.

Switch to guarded and compare the score.

Both candidates are deterministic Python functions, not trained models.

The project demonstrates evaluation mechanics, not model quality.

AI Evaluation Lab

Setup and repository map

Prerequisite: Python 3.10 or newer.

No pip install or API key is required.

Extract this ZIP, open a terminal in its project folder and run python server.py.

Open http://127.0.0.1:8000 in your browser.

If that port is busy, run python server.py --port 8001 and open port 8001 instead.

On Windows, py can replace python.

Stop with Ctrl+C.

Run python -m unittest discover -v to execute tests.

logic.py holds domain logic; server.py handles HTTP and local storage; index.html contains the browser UI; sample_data.json holds synthetic fixtures; test_logic.py checks domain behavior; test_http.py checks the HTTP boundary.

requirements.txt documents the standard-library-only runtime.

The guide-generation tools are not runtime dependencies.

AI Evaluation Lab

Architecture and technology choices

Browser form -> fetch POST /api/run -> Python HTTP handler -> domain run(payload, records) -> JSON result -> safe DOM tables.

GET / serves the app, GET /api/health returns a health response, and unknown paths return 404.

Python provides a portable runtime; http.server provides a minimal HTTP boundary; JSON keeps fixtures inspectable; HTML and CSS provide a responsive UI; JavaScript fetch connects the UI and backend; unittest provides repeatable checks.

Browser output uses textContent rather than HTML injection.

There is no framework, build system, external CDN, telemetry upload or database dependency.

AI Evaluation Lab

Scoring and data

Each sample case has id, question and expected.

A case passes when expected is a case-insensitive substring of the response.

The pass rate is passing cases divided by total cases; an empty dataset returns zero.

This is interpretable but brittle: a wrong response containing the expected words could pass.

Add semantic criteria, negative assertions and human labels before using this for real product decisions.

The sample has five synthetic cases and is not statistically representative.

Code walkthrough

logic.py contains answer() and run().

answer() implements two candidate strategies; run() iterates the dataset, times each call with perf_counter, grades it, and returns per-case evidence.

The guarded candidate checks a short keyword list before the knowledge lookup.

It is intentionally simple and can be bypassed.

test_logic.py asserts the known baseline regression, the guarded score, invalid input and empty-data behavior.

AI Evaluation Lab

Observability and real AI extension

The table records output and elapsed milliseconds per case.

These local timings are usually tiny and cannot predict hosted model speed.

To add a real model, replace answer() with a server-side provider adapter, read an API key from an environment variable, use timeouts, record model and prompt version, and retain token counts from the provider.

Never put credentials into browser JavaScript.

Add repeated runs and uncertainty estimates for nondeterministic responses.

Interview discussion and next steps

Explain why accuracy alone is insufficient: refusals, groundedness, latency, cost and user experience can conflict.

Demonstrate the failing baseline before the passing candidate.

Discuss false positive grades and why the guarded keyword filter is not a production security boundary.

Customize at least ten realistic cases, add a failure taxonomy, and implement one stronger rubric before presenting this as your own extension.

API, errors and operational limits

POST /api/run accepts a JSON object with Content-Type application/json.

The form fields below correspond to payload keys.

Successful calls return HTTP 200 and JSON; validation or malformed JSON returns 400; unknown paths return 404.

Bodies must be between 1 and 32768 bytes.

Unexpected local data or storage errors return 500 with a generic message.

The server binds to 127.0.0.1 and is intended for a trusted local demo.

Python http.server is not a production server.

Authentication, TLS, concurrent workers, rate limits and schema validation must be designed before deployment.

Do not expose this process directly to the internet.

Example request payload

```
{
  "candidate": "guarded"
}
```

AI Evaluation Lab

Testing, GitHub and ownership

Run the logic and HTTP tests before changing sample data, since expected values describe the shipped fixture.

The HTTP tests start a temporary local server and verify health, successful workflow, malformed JSON, non-object payloads and unknown paths.

This is a useful smoke suite, not comprehensive security or browser coverage.

Create a GitHub repository, copy the extracted project files into it, then run git init, git add ., git commit -m "Initial portfolio demo", git branch -M main, git remote add origin YOUR_REPOSITORY_URL, and git push -u origin main.

Replace the URL with your own empty repository.

Add a screenshot and a short video after running it.

This is an AI-assisted starter project; describe your own review, changes and learning accurately.

Do not claim customer usage, production deployment or measured model improvements from these synthetic examples.